AI Assisted Coding

Name : Vineeth-Chidurala **Date** : 01-04-2026

Ht.No. : 2303A52447

Task - 1 : Python to C++ Conversion
Translate a simple Python class into C++ using AI assistance.

Prompt : Convert the following Python class into an equivalent C++ class.

Requirements:

Class represents a Student with attributes: id, name, and marks

Include methods to display student details and calculate grade

Ensure proper use of:

Constructors

Data types

Access specifiers (public/private)

Follow object-oriented principles in C++

Provide complete working code with main function

Also ensure the output of both Python and C++ programs is the same.

Screenshots :

```python
    def calculate_grade(self):
        if self.marks >= 90:
            return 'A'
        elif self.marks >= 75:
            return 'B'
        elif self.marks >= 50:
            return 'C'
        else:
            return 'F'

    def display(self):
        print("ID:", self.student_id)
        print("Name:", self.name)
        print("Marks:", self.marks)
        print("Grade:", self.calculate_grade())


# Main
s1 = Student(101, "Vineet", 85)
s1.display()

# ------------------------------------------------------------------------------
#include <iostream>
using namespace std;

class Student {
private:
    int student_id;
    string name;
    float marks;

public:
    // Constructor
    Student(int id, string n, float m) {
```

[4] ✓ 0.0s Python

Output :

```
    // Method to calculate grade
    char calculateGrade() {
        if (marks >= 90)
            return 'A';
        else if (marks >= 75)
            return 'B';
        else if (marks >= 50)
            return 'C';
        else
            return 'F';
    }

    // Display method
    void display() {
        cout << "ID: " << student_id << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
        cout << "Grade: " << calculateGrade() << endl;
    }
};

// Main function
int main() {
    Student s1(101, "Vineet", 85);
    s1.display();
    return 0;
}
```

[4] ✓ 0.0s Python

```
ID: 101
Name: Vineet
Marks: 85
Grade: B
```

Task – 2 :Java to Python Function Conversion

Convert a Java method that checks whether a number is prime into Python.

Prompt : Convert the following Java method that checks whether a number is prime into Python.

Requirements:

Maintain the same logic and correctness

Use Pythonic syntax and indentation

Handle edge cases (numbers less than 2)

Provide a complete Python function

Include sample test cases to verify correctness

ScreenShots :

```java
public class PrimeCheck {
    public static boolean isPrime(int n) {
        if (n <= 1)
            return false;

        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0)
                return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isPrime(7));
    }
}
```

Output :

```
            if (n % i == 0)
                return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isPrime(7));
    }
}

#-----------------------------------------------------------------------

import math

def is_prime(n):
    if n <= 1:
        return False

    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True
```

Task-3 : Pseudocode to Python Implementation

Translate a given pseudocode algorithm into Python using AI.

Prompt : Convert the following pseudocode for sorting numbers into Python code.

Requirements:

Use proper Python syntax and indentation

Ensure the logic is correctly implemented

Make the program executable

Include sample test cases to verify correctness

Pseudocode (Bubble Sort):

Start, Input list of numbers

For i from 0 to n-1:
For j from 0 to n-i-2:
If arr[j] > arr[j+1]:
Swap arr[j] and arr[j+1]

Output sorted list

End

ScreenShots :

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr
# Test cases
lists = [
    [5, 2, 9, 1, 5, 6],
    [3, 0, -1, 8, 7],
    [10],
    []
]

for lst in lists:
    print("Original:", lst)
    print("Sorted:  ", bubble_sort(lst.copy()))
    print()
```

Output :

```python
]

for lst in lists:
    print("Original:", lst)
    print("Sorted:  ", bubble_sort(lst.copy()))
    print()
```

[6] ✓ 0.0s

```
Original: [5, 2, 9, 1, 5, 6]
Sorted:   [1, 2, 5, 5, 6, 9]

Original: [3, 0, -1, 8, 7]
Sorted:   [-1, 0, 3, 7, 8]

Original: [10]
Sorted:   [10]

Original: []
Sorted:   []
```

Task-4 : SQL to Pandas Query

Translate a SQL query into a Pandas DataFrame operation in

Python

Prompt : Convert the following SQL query into an equivalent Pandas

DataFrame operation in Python.

Requirements:

Use proper Pandas syntax

Ensure filtering and column selection are correctly implemented

Provide sample DataFrame for testing

Show output after execution

SQL Query:

SELECT name, salary

FROM employees

WHERE salary > 50000;

Screenshots :

```
SELECT name, salary
FROM employees
WHERE salary > 50000;

#-----------------------------------------------------------------------

import pandas as pd

# Sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
}

df = pd.DataFrame(data)

# Equivalent Pandas query
result = df[df['salary'] > 50000][['name', 'salary']]

print(result)
```

Output :

```
# Equivalent Pandas query
result = df[df['salary'] > 50000][['name', 'salary']]

print(result)
```

[7] ✓ 20.7s

```
...        name  salary
0     Alice   60000
2   Charlie   70000
```

Task-5 : Real-Time Application: Algorithm Translation Across Languages

A company maintains algorithms written in different programming languages.

Prompt : Translate a searching algorithm from Python to C.
Requirements:
Use Linear Search algorithm
Maintain the same logic in both languages
Ensure proper syntax conversion
Handle arrays and loops correctly
Provide complete working programs in both Python and C
Include test cases and output verification
Highlight challenges in translation (syntax, memory, libraries)

Screenshots :

```python
    return -1


# Test
arr = [10, 20, 30, 40, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")

#-----------------------------------------------------------------------

#include <stdio.h>

int linearSearch(int arr[], int size, int key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int size = sizeof(arr) / sizeof(arr[0]);
```

Output :

```c
    if (result != -1)
        printf("Element found at index: %d\n", result);
    else
        printf("Element not found\n");

    return 0;
}
```

[8] ✓ 0.0s

```
Element found at index: 2
```